

FINAL PROJECT PROPOSAL

WatchVault

3-29-2026

PREPARED FOR

SE/COM S 3190 - Construction of User Interfaces

Iowa State University

Department of Computer Science

PREPARED BY

Alex Erickson

Octavio Munoz

Contents

- 1. Introduction**
- 2. Purpose of the proposal**
- 3. Goals and Objectives**
- 4. Project Description**
- 5. Detailed Description of Pages**
- 6. Exercising Project Options**
- 7. Feature Ownership & Responsibility**
- 8. Resources and Tools**
- 9. Data Sources and Management**
- 10. Authentication System**
- 11. System Architecture Overview**
- 12. User Experience Views**
- 13. Error Handling and Edge Cases**
- 14. Final Comments**

1 Introduction

We, Alex Erickson and Octavio Munoz, are from team AL_10. Alex is a student majoring in Software Engineering. Octavio is a Computer Science major. We are learning demanding web development skills such as HTML, CSS, and JavaScript in this SE/COM S 3190 class. We learned how to build a website for a product catalog through our in-class activities. But we also have prior experience in website development through personal projects or internships.

2 Purpose of Proposal

We aim to develop a user-friendly website that is a good place to track or create a movie watchlist. Through our website “WatchVault,” we hope to address the problem of users who enjoy films often lose track of movies they want to watch or have already seen. WatchVault solves this by making a website that users can browse, search, and filter a catalog of movies. Each movie will include detailed information.

3 Goals and Objectives

Our project’s goals and objectives include:

- Develop a visually appealing, user-friendly movie browsing application that feels intuitive and polished.
- Provide a large enough movie catalog so users can build a meaningful, personalized watchlist.
- Demonstrate proficiency in Full-stack web application using modern technologies
- Populate the application with a minimum of 20–30 movies stored in a structured Database like MongoDB, each with a title, genre, year, rating, synopsis, and cast.
- Implement a live search bar and a genre/sort filter that dynamically update the displayed results as the user enters input.
- Build a multi-view interface with straightforward navigation between the Different Pages.
- Allow users to add and remove movies from a personal watchlist, with the list updating in the UI immediately.
- Display a dedicated Movie Detail view for each title, showing complete information when a card is selected.
- Include basic error handling, such as an empty-state message when no search results are found.
- Implement RESTful APIs supporting GET, POST, PUT, and DELETE operations

Objectives

- Week 1: Get Base Project structure started
- Week 2: Develop Frontend UI

- Week 3: Implement API's, Backend logic, and database
- Week 4: Connect both frontend and back end
- Week 5: Test, debug, and finalize

4 Project

With our project "WatchVault," we intend to create a website for movie watchers. Our goal is to create a website that allows users to make a movie watchlist or a list of movies they have already watched.

There will be two main roles present

- User: An authenticated user stored in the database that can save a watchlist and any other data the website has.
- Admin: Maintain API keys and movie catalogs

Our Project will feature several key components:

- Homepage – Displays featured movies, main navigation, and a quick-search bar for easy entry.
- Search Page – Offers a prominent search field, filter options, and a grid of result thumbnails.
- Watchlist Page – Shows the user's saved titles with thumbnail previews and quick-remove controls.
- Recommendations Page – Presents a curated carousel of suggested movies with brief info cards.
- Login Page – Provides a simple sign-in form with email/password fields and links for recovery or registration.
- Movie Detail Page – Highlights the poster, synopsis, cast, rating, and actions such as play or add to watchlist.
- Review and Ratings Page – Lists user reviews and star ratings while allowing new comments to be submitted.
- Settings Page – Contains account preferences, notification toggles, theme selection, and privacy options.
- Admin Page - Allows Admin user to manage api keys and movie meta data
- A good-sized library of movies of different genres

5 Detail Description of Pages

To provide a clear understanding of the system from a user and client perspective, we describe each page of the application in detail below. Each page is designed to serve a specific purpose and is connected through a consistent navigation structure.

- The Homepage serves as the main entry point for users, displaying a selection of featured movies along with visually engaging banners or thumbnails. It includes the primary navigation menu for accessing other sections of the application, as well as a

quick-search bar that lets users look up movies without leaving the page. The layout is designed to be clean and intuitive, helping users quickly discover trending or recommended content.

- The Search Page provides users with a powerful and flexible way to find movies. It features a prominent search field where users can enter titles, genres, or keywords, along with filter options such as release year, rating, or category to refine results. Search results are displayed in a grid format with thumbnail images, making it easy for users to browse and select movies of interest.
- The Watchlist Page allows users to keep track of movies they want to watch later. It displays a personalized list of saved titles, each with a thumbnail preview and basic information. Users can quickly remove items from their watchlist using simple controls, making it easy to manage and update their saved content.
- The Recommendations Page is designed to enhance user engagement by offering a curated selection of movie recommendations. These recommendations may be based on user preferences, viewing history, or trending content. The page typically uses a carousel or scrollable layout, with each movie accompanied by a brief information card that includes genre, rating, and a short description.
- The Login Page provides a secure and straightforward interface for user authentication. It includes input fields for email and password, along with options for password recovery and new user registration. The design emphasizes simplicity and accessibility, ensuring users can quickly sign in or create an account.
- The Movie Detail Page offers an in-depth view of a selected movie. It prominently displays the movie poster along with key information such as the synopsis, cast members, release details, and overall rating. Users are also given actionable options, such as playing the movie (if available) or adding it to their watchlist, making this page central to user interaction.
- The Review and Ratings Page allows users to engage with the community by reading and contributing feedback. It displays a list of user-submitted reviews and star ratings, providing insight into others' opinions. Additionally, users can submit their own reviews and ratings, encouraging interaction and discussion around specific movies.
- The Settings Page gives users control over their account and application preferences. This includes options for managing personal information, adjusting notification settings, selecting visual themes (such as light or dark mode), and configuring privacy options. The goal is to provide a customizable and user-friendly experience.
- The Admin Page is designed for administrative users to manage the application's backend. It allows admins to securely handle API keys, ensuring proper integration with external movie data services. Additionally, admins can manage movie metadata, including updating titles, descriptions, genres, and other relevant information. This page is essential for maintaining accurate data and ensuring the system operates smoothly.
-

6 Exercising Project Options

We have selected Option 1: Extending the Midterm Project.

This project is a continuation of our midterm work, redesigned using a full-stack architecture. We will:

- Convert static pages into dynamic React components.
- Build REST APIs using Node.js and Express.
- Store and manage data using a database system.
- Ensure all data displayed on the frontend is fetched dynamically from the backend.

This approach allows us to enhance our previous work while introducing new technical concepts.

7 Feature Ownership & Responsibility

Each member will take end-to-end ownership of specific features, handling both frontend and backend components. To ensure equal contribution and proper coverage of all REST API operations, the project is divided such that each member is responsible for four pages and two complete features, with each feature supporting GET, POST, PUT, and DELETE operations.

Member 1 (Admin & Core Data Management)

Pages Owned:

- Search Page
- Movie Detail Page
- Review and Ratings Page
- Admin Page

Feature 1: Movie Catalog Management

- GET: Retrieve all movies for the search page and general listings
- POST: Add a new movie (admin only)
- PUT: Update existing movie details
- DELETE: Delete a movie from the system

Feature 2: Movie Metadata, Reviews, & Ratings Management

- GET: Retrieve detailed movie information, including reviews and ratings
- POST: Add movie metadata, reviews, or ratings
- PUT: Update movie metadata, reviews, or ratings
- DELETE: Remove movie metadata, reviews, or ratings

Responsibility Scope:

Member 1 will focus on backend-heavy features and ensure that all movie-related data is

correctly stored, retrieved, and updated. This includes managing API integrations, handling review and rating data, maintaining database structure, and ensuring accurate and efficient data flow to the frontend pages.

Member 2 (User Interaction & Experience)

Pages Owned:

- Homepage
- Watchlist Page
- Recommendations Page
- Login Page
- Settings Page

Feature 1: Watchlist Management

- GET: Retrieve the user's watchlist
- POST: Add a movie to the watchlist
- PUT: Update watchlist preferences (e.g., notes or priority)
- DELETE: Remove a movie from the watchlist

Feature 2: User Account & Personalization

- GET: Retrieve user profile, preferences, and recommendations
- POST: Create user accounts and save preferences
- PUT: Update account settings, themes, and notification preferences
- DELETE: Remove user data or deactivate account

Responsibility Scope:

Member 2 will focus on user-facing features to ensure a smooth, intuitive experience. This includes designing and implementing UI components, handling authentication and session management, and integrating backend APIs to provide personalized content such as recommendations and watchlists.

This structured division ensures that both members contribute equally while also gaining experience across the full stack. Each feature is implemented independently but integrates seamlessly into the overall system, allowing for efficient development, testing, and debugging.

8 Feature Ownership & Responsibility

Technologies and Tools:

- Frontend: React, React-Bootstrap, React Router
- Backend: Node.js, Express
- Database: MongoDB
- Design Tools: Excalidraw for wireframes

- Version Control: GitLab with milestones and issue tracking
- Testing Tools: Postman and browser developer tools

Each team member will contribute approximately 8–10 hours per week toward the project. Regular meetings will be conducted to track progress, discuss development updates, and resolve any issues that arise during implementation.

9 Data Sources and Management

Primary Data Source:

All application data will be stored in a database system such as MongoDB. The database will support both user-facing features and admin-level management, ensuring efficient storage, retrieval, and updates of movie and user data.

Data Entities:

- Movies: Stores core movie information, including title, genre, release year, description, and poster.
- Users: Stores account information, including login credentials, preferences, and settings.
- Watchlist: Stores movies that users have saved for later viewing.
- Reviews and Ratings: Stores user-submitted reviews, comments, and star ratings for each movie.
- Recommendations: Stores or generates personalized movie suggestions based on user activity and preferences.

Data Flow:

1. The user interacts with the frontend (e.g., searching for a movie, adding to watchlist, or submitting a review).
2. A request is sent to the backend API through defined routes.
3. The backend processes the request, applies business logic, and interacts with the database to retrieve or update data.
4. The backend sends a response back to the frontend.
5. The frontend dynamically updates the UI to reflect the latest data (e.g., displaying search results, updated watchlists, or new reviews).

10 Data Sources and Management

To ensure secure and personalized access to the movie application, a basic authentication system will be implemented. This system allows users to create accounts, log in, and access user-specific features such as watchlists, recommendations, reviews, and settings.

User Authentication Features:

- Signup: New users can create an account by providing details such as name, email, and password.
- Login: Existing users can log in using their registered credentials.
- Logout: Users can securely log out of the application.

Authentication Flow:

1. The user enters their credentials on the Login or Signup page.
2. The frontend sends a request to the backend API.
3. The backend validates the credentials and stores or retrieves user data from the database.
4. Upon successful authentication, the system generates a session or token.
5. The frontend stores this token and uses it to authenticate future API requests.

Security Considerations:

- Passwords will be securely stored using hashing techniques (e.g., bcrypt).
- Only authenticated users can access personalized features such as watchlists, reviews, recommendations, and settings.
- Admin routes (such as the Admin Page) will be restricted to authorized users only.

Integration with Features:

- The Watchlist Page, Review and Ratings Page, and Recommendations Page are accessible only to logged-in users.
- The Admin Page requires authentication with elevated privileges.
- All user-specific data is linked to unique user IDs stored in the database.

This authentication system ensures the application remains secure while providing personalized experiences and proper access control.

11 System Architecture Overview

The system follows a standard client-server architecture, where the frontend, backend, and database work together to deliver a complete movie browsing experience.

Architecture Components:

- Frontend (React): Handles the user interface, navigation, and interactions across pages such as Homepage, Search, Watchlist, and Movie Details.
- Backend (Node.js & Express): Processes API requests, applies business logic, manages authentication, and communicates with the database.
- Database (MongoDB): Stores movie data, user accounts, watchlists, reviews, ratings, and metadata.

System Flow:

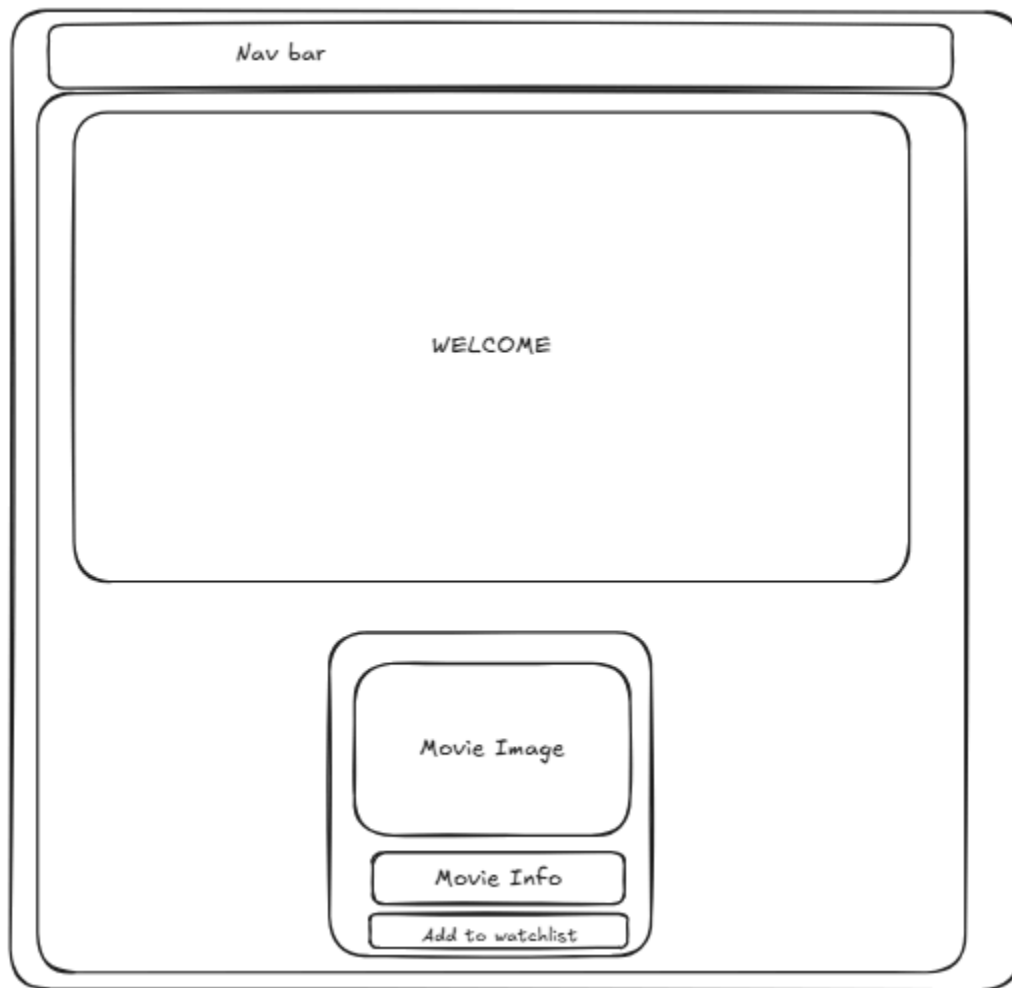
1. The user interacts with the frontend (e.g., searching for a movie, viewing details, or adding to a watchlist).
2. The frontend sends a request to the backend API.
3. The backend processes the request and interacts with the database.
4. The database returns the requested data.
5. The backend sends a JSON response back to the frontend.
6. The frontend dynamically updates the UI to reflect the changes.

This architecture ensures scalability, separation of concerns, and efficient data handling.

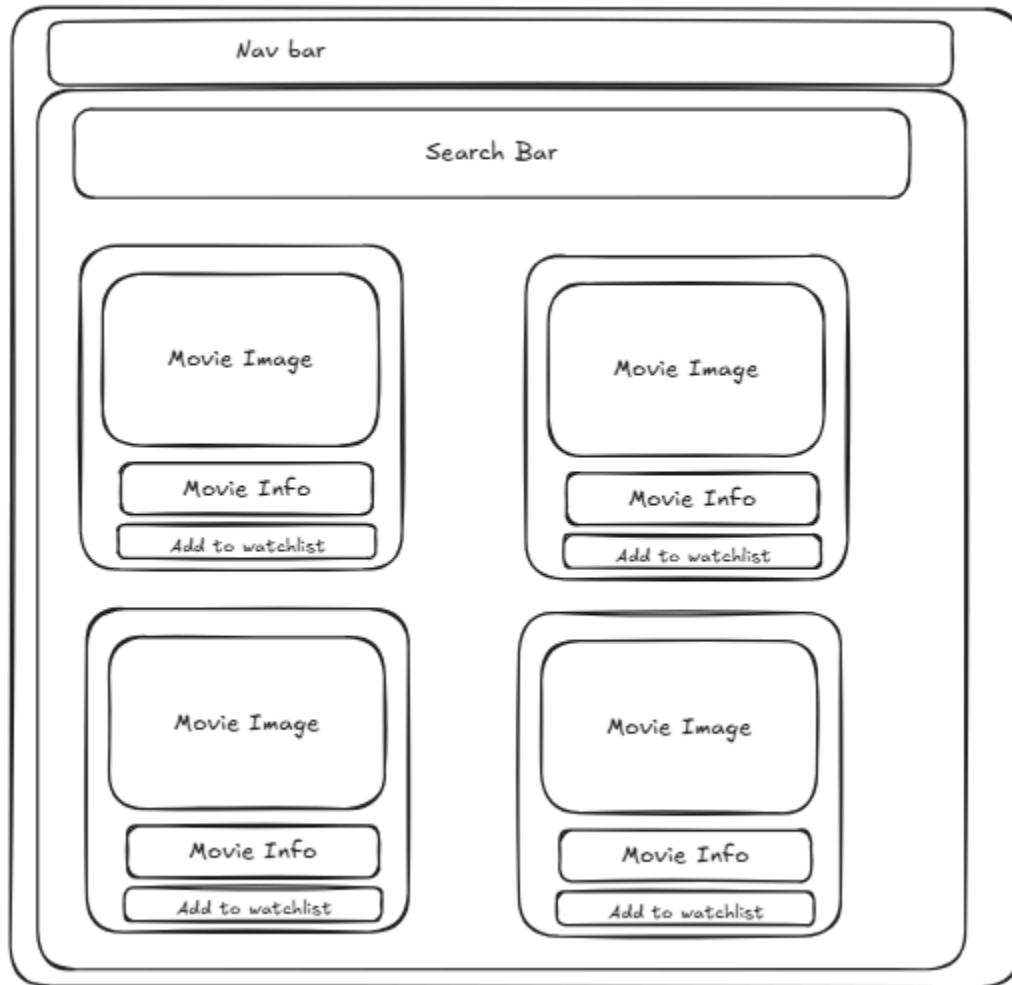
12 User Experience Views

Wireframes for each page will be created using Excalidraw. Each page is designed with a focus on clarity, usability, and smooth navigation. The goal is to ensure that users can easily understand and interact with the system without confusion.

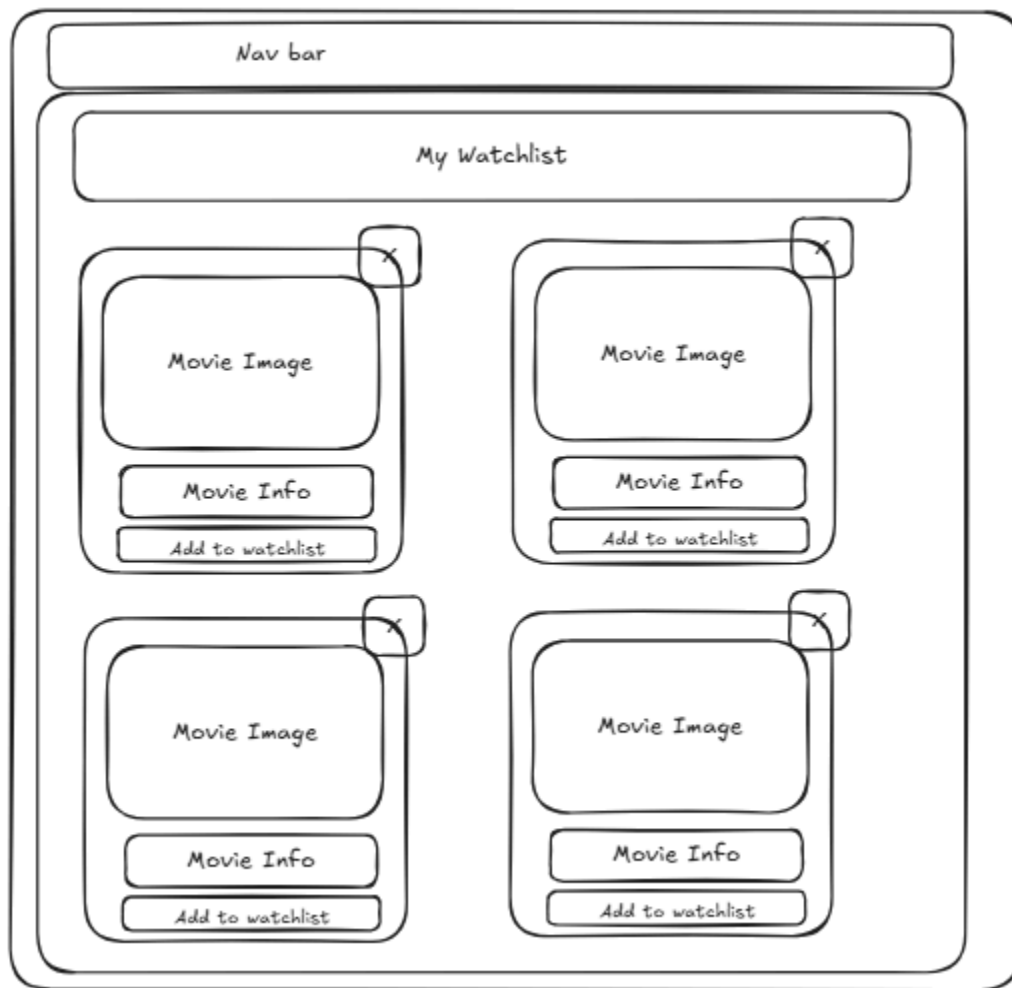
- Homepage



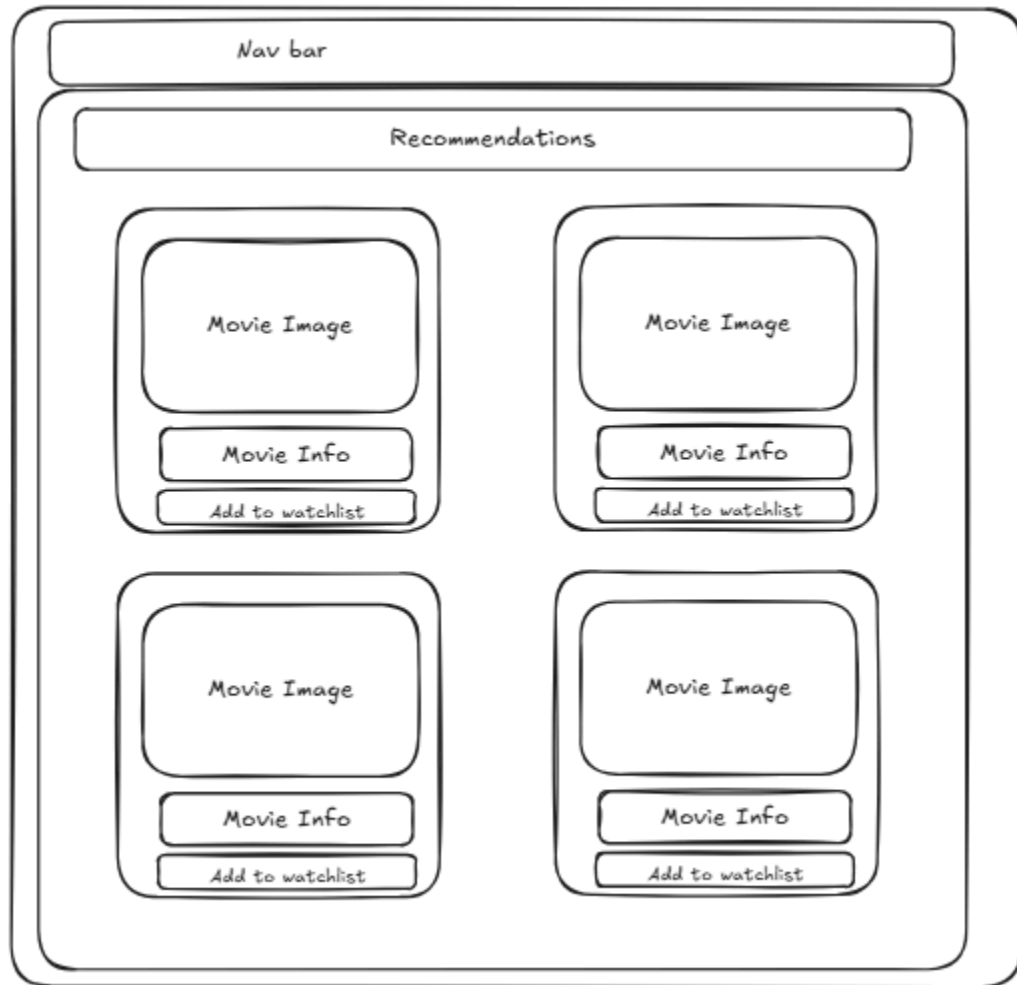
- Search Page



- Watchlist Page



- Recommendations Page



- Login Page

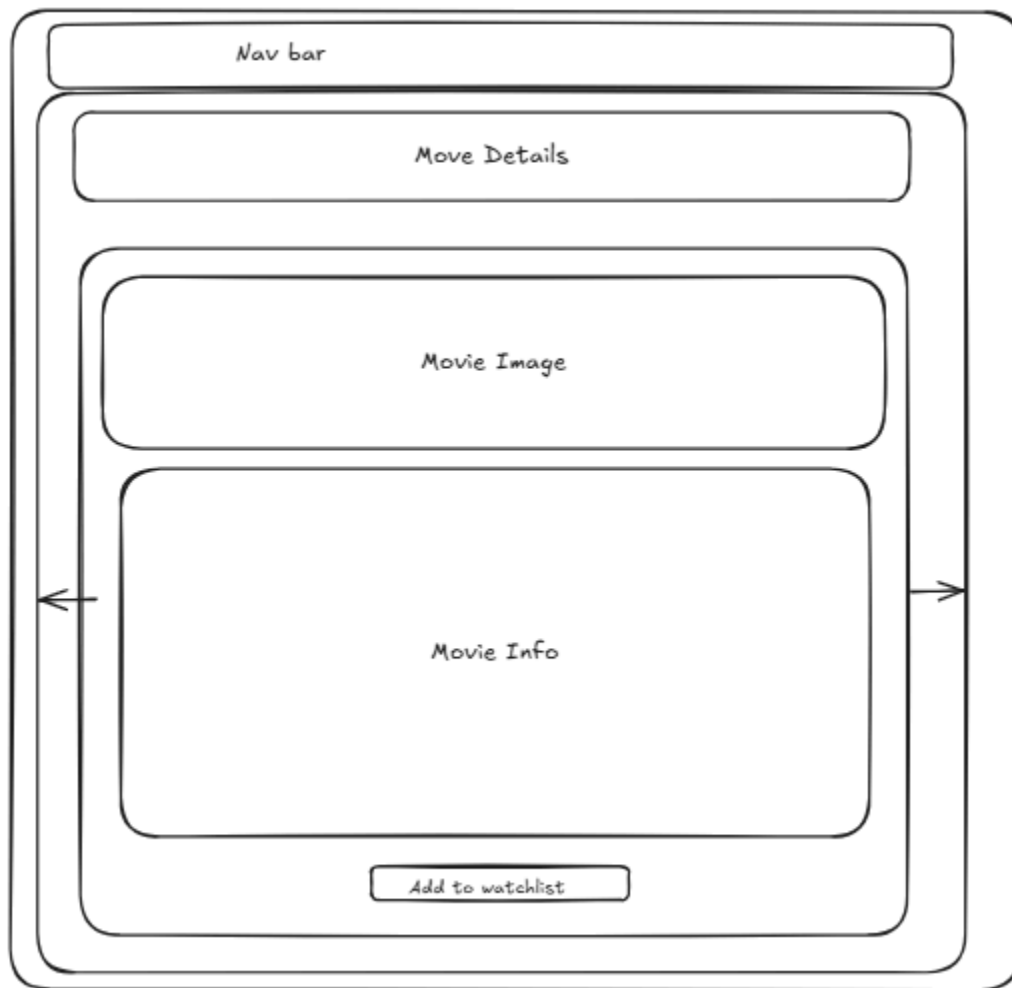
Nav bar

USERNAME

PASSWORD

LOGIN

- Movie Detail Page



- Review and Ratings Page

The wireframe illustrates a 'Review and Ratings' page layout. It features a main container with rounded corners. At the top is a 'Nav bar' section. Below it is a 'Move Details' section. The main content area contains eight horizontal bars, each labeled 'Rating'. The bottom-most bar is labeled 'Enter Rating'.

Nav bar

Move Details

Rating

Rating

Rating

Rating

Rating

Rating

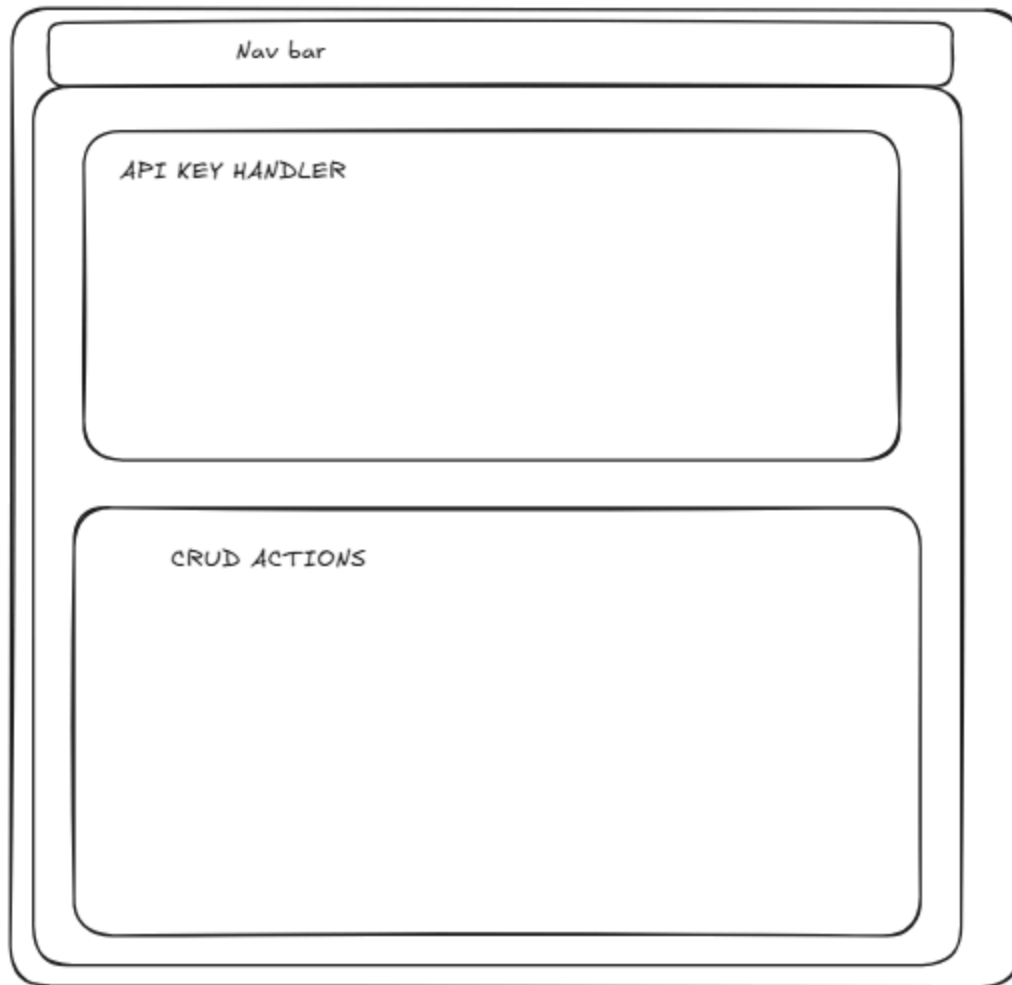
Rating

Enter Rating

- Settings Page



-
- Admin Page



All pages will follow a consistent design style and will be connected through a shared navigation bar to provide a seamless user experience across the application.

13 Error Handling and Edge Cases

To ensure the system is reliable and user-friendly, potential edge cases and errors will be handled throughout the application.

- Invalid Inputs: Forms such as login, signup, reviews, and admin movie creation will validate user input to prevent empty or incorrect data.
- Duplicate Entries: The system prevents duplicate movies and duplicate entries in watchlists or reviews.
- API Failures: Clear error messages will be displayed if the backend fails to respond or if network issues occur.

- Unauthorized Access: Users will not be able to access protected routes such as the Admin Page without proper authentication.

These measures improve system stability and enhance the overall user experience.

14 Final Comments

This is our proposal for WatchValut. This project will not only enhance our learning but also provide a good website for movie watchers to track or create watchlists of upcoming and previously watched movies.

You can reach out to us at: aerick22@iastate.edu or munozo@iastate.edu